# ESS PORTAL

## Tech Stack Requirements:

• **Frontend**: Next.js (App Router), React, Tailwind CSS

• **Backend**: Node.js with Express

• **Database**: PostgreSQL (using an ORM like Prisma or raw SQL queries)

• **Authentication**: JSON Web Tokens (JWT) for secure role-based session management

## Database Schema Setup:

Generate a migration or setup script for PostgreSQL with the following three essential tables:

1. 'Employees' Table: id, name, email, password_hash, role (ENUM: 'Employee', 'Manager'), department_id, leave_balance (default 21).

2. 'Attendance' Table: id, employee_id, date, clock_in_time, clock_out_time, total_hours.

3. 'Leaves' Table: id, employee_id, leave_type, start_date, end_date, status (ENUM: 'Pending', 'Approved', 'Rejected'), approved_by (manager_id).

## Backend API Implementation (Node.js/Express):

• Build a secure, modular REST API with the following endpoints protected by JWT role verification:

• POST /api/auth/login -> Validates user credentials and returns a JWT token containing user ID and role.

• POST /api/attendance/clock-in -> Records the current timestamp for the logged-in employee.

• POST /api/attendance/clock-out -> Records the clock-out timestamp and calculates total_hours for the day.

• GET /api/attendance/summary -> Retrieves attendance history for the

logged in user.

• POST /api/leaves/apply -> Allows an employee to submit a leave request specifying start_date, end_date, and leave_type. Deducts balance only upon approval.

• GET /api/leaves/pending -> (Manager Only) Fetches all pending leave requests submitted by employees.

• PUT /api/leaves/respond/:id -> (Manager Only) Updates a leave status to 'Approved' or 'Rejected'.

## Frontend Interface Implementation (Next.js & Tailwind CSS):

Create a clean, scannable, mobile-responsive dashboard containing three main views based on role routing:

1. **Login Page**: A secure form managing JWT token assignment and storage.

2. **Employee Dashboard View**:

• A Quick Actions component with a prominent "Clock In / Clock Out" toggle button.

• A simple form to "Apply for Leave" (date pickers and dropdown for type). - Summary cards displaying current Leave Balance and a log of past Attendance hours.

3. **Manager Approval View** (Conditional Route):

• A dedicated dashboard section visible only to accounts with the 'Manager' role.

• A clean UI table displaying pending leave requests with immediate "Approve" and "Reject" action buttons that trigger API updates and refresh the view.

# Internship Management, Collaboration & Mentorship Portal

## Tech Stack Requirements:

- **Frontend**: Next.js (App Router), React, TypeScript, Tailwind CSS
- **Backend**: Node.js with Express (Modular Routing Structure)
- **Database**: PostgreSQL (using Prisma ORM or raw SQL queries)
- **Authentication**: JSON Web Tokens (JWT) for secure role-based session management
- **Real-Time Communication**: Socket.IO
- **File Storage**: Cloudinary or AWS S3

## Database Schema Setup:

Generate a migration or setup script for PostgreSQL with the following core tables:

1. **Users Table:** id, name, email, password_hash, role (ENUM: 'Intern', 'Mentor', 'Admin'), mentor_id, created_at.
2. **Tasks Table:** id, title, description, deadline, priority (ENUM: 'Low', 'Medium', 'High'), status (ENUM: 'To-Do', 'In-Progress', 'Under-Review', 'Completed'), assigned_to, created_by.
3. **Submissions Table:** id, task_id, intern_id, submission_text, repository_url, submitted_at, feedback_text, rating.

## Backend API Implementation (Node.js/Express):

- POST /api/auth/login → Validates user credentials and returns JWT token.
- GET /api/tasks → Fetches assigned tasks for Intern or created tasks for Mentor.
- POST /api/tasks/create → (Mentor Only) Creates and assigns tasks.
- PUT /api/tasks/:id/status → Updates task status.
- POST /api/submissions/submit → Submit completed work.
- PUT /api/submissions/:id/review → (Mentor Only) Reviews and rates submissions.

## Frontend Interface Implementation (Next.js & Tailwind CSS):

1. **Login Page**
    - Secure login form.
    - JWT token storage.
2. **Intern Dashboard**
    - Assigned Tasks Summary.
    - Progress Tracking Cards.
    - Task Status Updates.
    - Submission Form.
3. **Mentor Dashboard**
    - Task Creation Panel.
    - Assigned Intern Overview.
    - Pending Submission Reviews.
    - Performance Rating Interface.

# Video Communication & Virtual Collaboration Platform

## Tech Stack Requirements:

• **Frontend**: Next.js (App Router), React, TypeScript, Tailwind CSS
• **Backend**: Node.js with Express
• **Real-Time Communication**: Socket.IO
• **Video Engine**: WebRTC
• **Database**: PostgreSQL (Prisma ORM)
• **Authentication**: JWT

## Database Schema Setup:

Generate a PostgreSQL schema with the following tables:

1. **Rooms Table:** id, room_slug, created_at, is_active.

2. **Participants Table:** id, room_id, user_name, joined_at, left_at.
3. **Messages Table:** id, room_id, sender_name, message_text, sent_at.

## Backend API Implementation (Node.js/Express):

• POST /api/rooms/create → Creates a new meeting room.
• GET /api/rooms/:id → Fetch room details.
• POST /api/auth/login → User authentication.

## Socket.IO Events:

• join-room
• offer
• answer
• ice-candidate
• send-message
• leave-room

## Frontend Interface Implementation (Next.js & Tailwind CSS):

1. **Landing Page**
    - Create Meeting Button.
    - Join Meeting Input.
2. **Meeting Room View**
    - Local Video Feed.
    - Remote Video Feed.
    - Audio/Video Controls.
    - Screen Sharing Button.
3. **Chat Sidebar**
    - Real-time Messaging.
    - Participant List.
    - Shared Links Feed.

# Enterprise Work Management & Collaboration Platform

## Tech Stack Requirements:

- **Frontend**: Next.js (App Router), React, TypeScript, Tailwind CSS
- **Backend**: Node.js with Express
- **Database**: PostgreSQL (Prisma ORM)
- **Authentication**: JWT

## Database Schema Setup:

Generate a PostgreSQL schema with the following core tables:

1. **Users Table:** id, name, email, password_hash, role.
2. **Workspaces Table:** id, name, created_by.
3. **Projects Table:** id, workspace_id, name, description.
4. **Tasks Table:** id, project_id, title, description, due_date, priority, status, assigned_to.
5. **Comments Table:** id, task_id, user_id, comment_text, created_at.

## Backend API Implementation (Node.js/Express):

- POST /api/auth/login → User authentication.
- GET /api/workspaces → Fetch workspaces.
- GET /api/projects → Fetch projects.
- GET /api/tasks → Fetch project tasks.
- POST /api/tasks/create → Create task.
- PUT /api/tasks/:id/status → Update task status.
- POST /api/tasks/:id/comments → Add comments.

## Frontend Interface Implementation (Next.js & Tailwind CSS):

1. **Workspace Dashboard**
    - Workspace Navigation Sidebar.
    - Project Explorer.
2. **Kanban Board**
    - To-Do Column.

- In-Progress Column.
- Done Column.
- Drag & Drop Support.

3. **Task Details Panel**
    - Description View.
    - Due Date.
    - Comment Section.
    - Assigned User Information.

# Microsoft Teams Replica

## Tech Stack Requirements:

• **Frontend**: Next.js (App Router), React, TypeScript, Tailwind CSS
• **Backend**: Node.js with Express
• **Database**: MongoDB (Mongoose ODM)
• **Authentication**: JWT
• **Real-Time Communication**: Socket.IO

## Database Schema Setup:

Generate MongoDB schemas for the following collections:

1. **Users Collection:** id, name, email, password_hash, presence_status, assigned_teams.
2. **Teams Collection:** id, name, description, members.
3. **Channels Collection:** id, team_id, name, type.
4. **Messages Collection:** id, sender_id, recipient_type, recipient_id, text_content, file_url, created_at.

## Backend API Implementation (Node.js/Express):

• POST /api/auth/login → User authentication.
• GET /api/teams → Fetch assigned teams.
• GET /api/channels/:id/messages → Retrieve channel messages.
• POST /api/messages/upload-file → Upload attachments.

## Socket.IO Events:

- register-user
- presence-change
- join-channel-room
- chat-message
- typing-indicator

## Frontend Interface Implementation (Next.js & Tailwind CSS):

1. **Workspace Sidebar**
    - Teams List.
    - Channels List.
    - Presence Status Selector.
2. **Direct Messages Panel**
    - Recent Conversations.
    - Online Status Indicators.
3. **Chat Window**
    - Message Feed.
    - File Sharing.
    - Typing Indicators.
    - Message Input Box.
4. **Channel Workspace**
    - Channel Discussions.
    - Shared Files.
    - Team Announcements.